Software Design Analysis Report

A. Project Details

Project title	Image Forgery Detection System (detects and localizes tampered regions such as copy-move and splicing in digital images)
Team / roll numbers	Keertivanth - 202401100100133 Khushhal - 202401100100134 Ayush Raj - 202401100100083 Atul Kumar Agrahari - 202401100100074
Project objective	To build a web-based system where a user uploads an image and the system decides whether it is authentic or forged using Error Level Analysis (ELA) and a CNN classifier. For forged images it also highlights the tampered region with a heatmap and produces a downloadable report.
Technology stack	Python, Flask (REST API), OpenCV, NumPy, TensorFlow/Keras (CNN), HTML/CSS/JavaScript (front end), SQLite/MySQL (database), ReportLab (PDF reports)

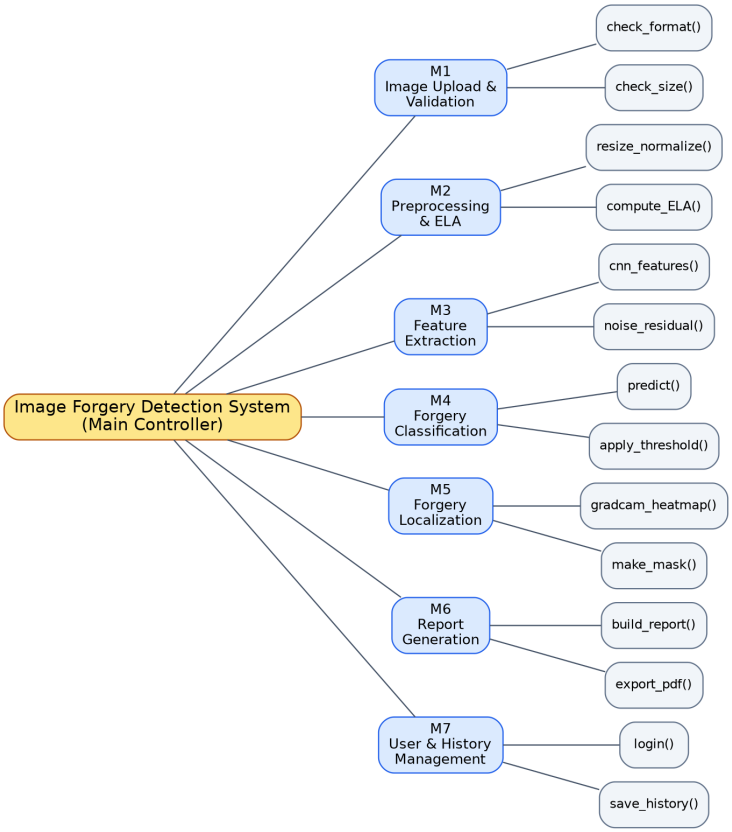
Note: the project was assumed to be image forgery detection; edit the content if your scope differs (e.g., signature or document forgery).

B. Module Identification and Modularization

Total number of modules: 7. The system is divided along the stages of the detection pipeline (input, preparation, analysis, decision, localization, output) plus one supporting module for users and storage. Each stage has one clear job, so modules can be developed, tested and replaced independently (for example, the CNN can be swapped without touching upload or reporting code).

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	Image Upload & Validation	Accept image, check format (JPG/PNG) and size	In: image file. Out: validated image	M7 (session)
M2	Preprocessing & ELA	Resize, normalize and generate Error Level Analysis image	In: validated image. Out: tensor + ELA image	M1
M3	Feature Extraction	Extract CNN features from the ELA/RGB tensor	In: tensor. Out: feature vector	M2
M4	Forgery Classification	Predict Authentic/Forged with confidence score	In: feature vector. Out: label, confidence	M3
M5	Forgery Localization	Create Grad-CAM heatmap and tampered-region mask	In: image, prediction. Out: heatmap, mask	M2, M4
M6	Report Generation	Build result summary and export PDF	In: result, heatmap. Out: report file	M4, M5
M7	User & History Management	Login, store and retrieve past results	In: credentials, result. Out: session, history	Database

Module hierarchy / decomposition diagram:



C. Design Strategy

Selected strategy: [ ] Top-Down [ ] Bottom-Up **[X] Hybrid**

Justification: The overall flow of the system (upload, analyse, decide, report) is clear from the start, so the design began top-down by splitting the whole system into modules. However, the core detection part depends on ready-made, reusable components such as OpenCV image functions, an ELA routine and a pretrained CNN backbone (e.g., VGG/ResNet), which are best built and tested bottom-up first. A hybrid strategy lets us decompose the problem clearly while reusing tested components and integrating them step by step, which reduces risk and development time.

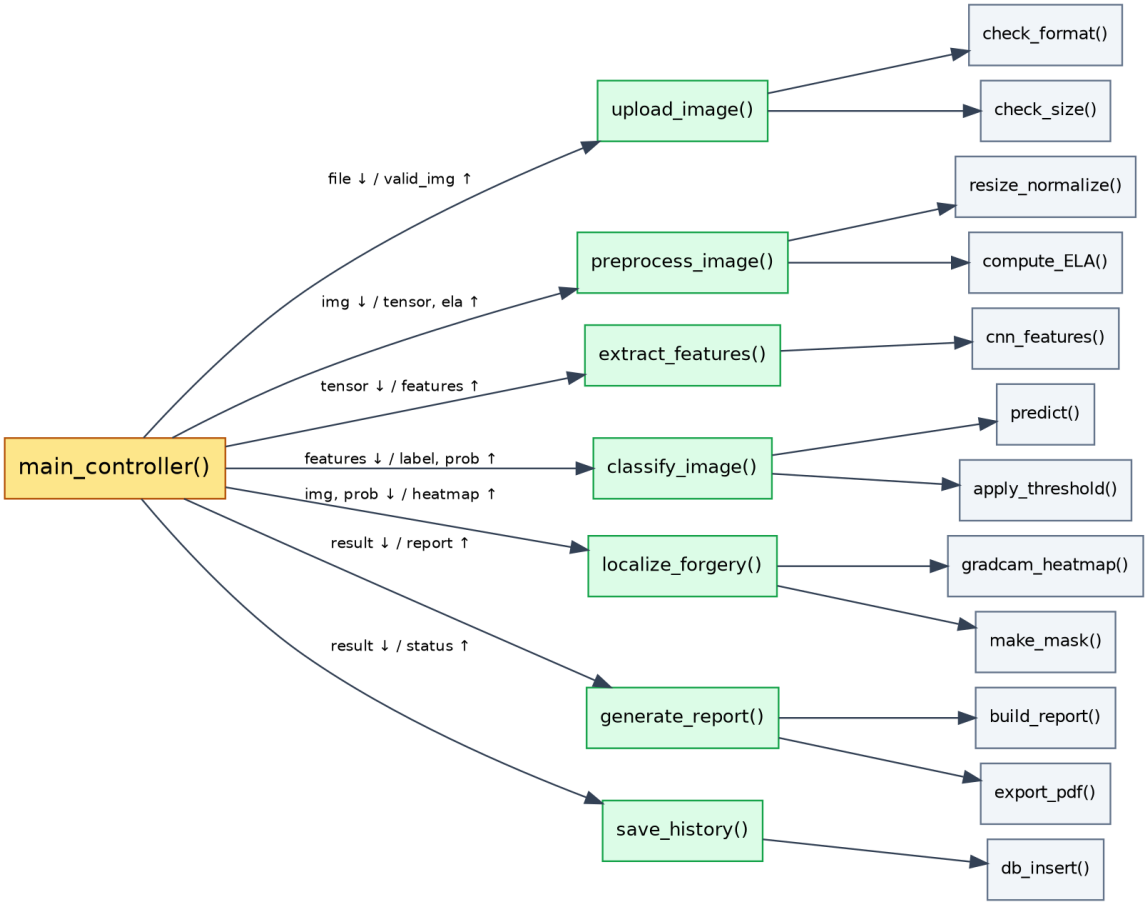
Evidence: Top-down: the system was first defined as one goal, 'detect forgery in an image', then broken into 7 modules (M1-M7) and further into functions (see decomposition above). Bottom-up: low-level units (compute_ELA, resize_normalize, predict) were coded and unit-tested first, then combined into M2, M3, M4 and finally integrated with M1, M5, M6 and M7 through the main controller.

D. Design Approach

Selected approach: [ ] Function-Oriented [ ] Object-Oriented **[X] Mixed**

Justification: The detection pipeline is naturally a sequence of functions (validate, resize, compute_ELA, extract_features, predict, make_heatmap), so a function-oriented design suits the processing part. The data and actors of the system (User, ImageFile, DetectionResult, Report) are real-world entities with attributes and behaviour, so they are modelled as classes. Main functions: check_format(), compute_ELA(), predict(), gradcam_heatmap(), export_pdf(). Main classes: User, ImageFile, Preprocessor, ForgeryClassifier, DetectionResult, ReportGenerator (see UML class diagram).

Evidence - Structure chart: The main controller calls each module function; downward labels show data passed and upward labels show data returned.



Explanation: main_controller() coordinates the pipeline and is the only function that knows the order of execution. Each called function receives only the data it needs (data coupling) and returns a result, so modules do not call each other directly.

E. Coupling Analysis

Aim: low coupling. All interactions below pass only the required data through the controller.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M1 - M2	Validated image (array/file path)	Data coupling (low)	Pass only the image, not the upload object or user session
M2 - M3	Preprocessed tensor of fixed size	Data coupling (low)	Keep a fixed tensor shape contract so the extractor can change freely
M3 - M4	Feature vector	Data coupling (low)	Use a plain array interface; classifier unaware of how features are built
M4 - M5	DetectionResult object (label, confidence, layer outputs)	Stamp coupling (moderate)	Pass only label and confidence fields needed by the localizer
M5 - M6	Result object + heatmap path	Stamp coupling (moderate)	Use a small DTO with only report fields

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
M6 - M7	Result record to store	Data coupling (low)	Access the database only through HistoryRepository methods

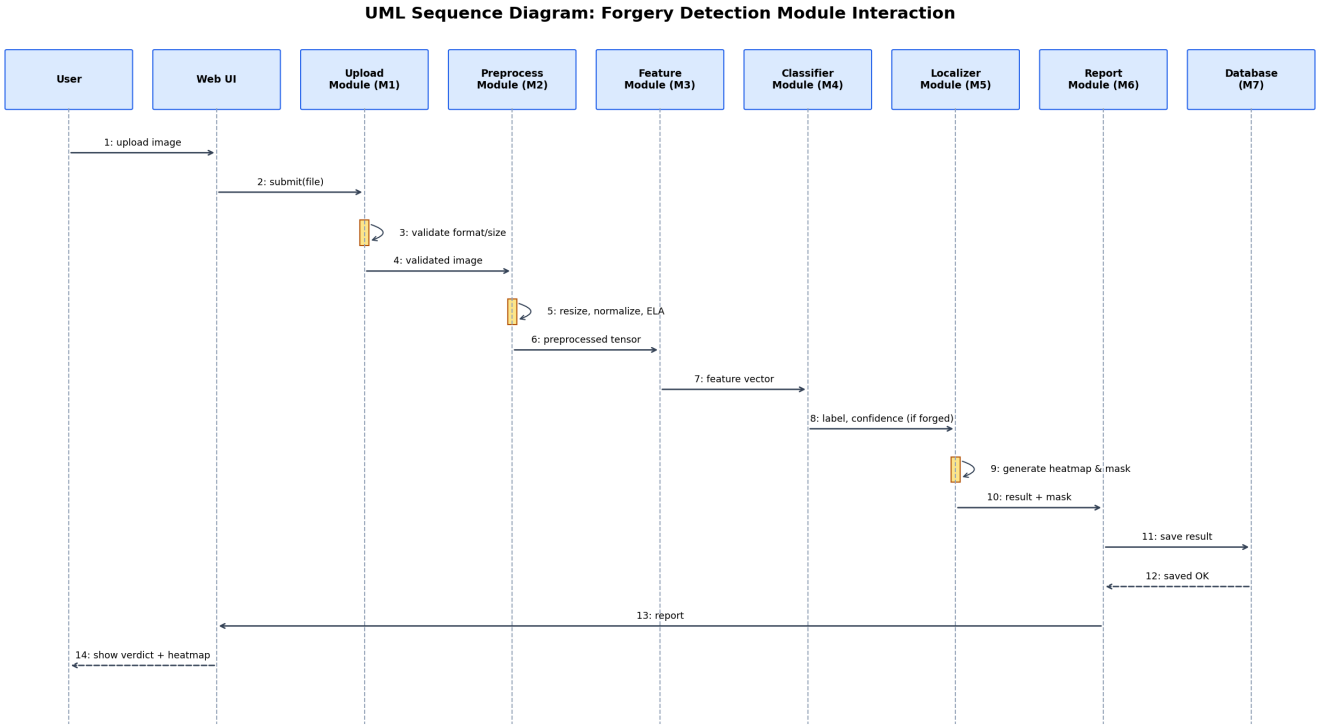
F. Cohesion Analysis

Aim: high cohesion. Each module has one main responsibility.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
M1 Upload & Validation	Check that the uploaded file is an acceptable image	Functional (high)	None; keep storage of files in a separate helper
M2 Preprocessing & ELA	Convert an image into model-ready input	Sequential (high): output of resize feeds ELA	Split resize and ELA into separate functions (already done)
M3 Feature Extraction	Produce feature vector from tensor	Functional (high)	None
M6 Report Generation	Build and export the report	Sequential/Functional (high)	Keep database saving out of this module (handled by M7)
M7 User & History	Login and history records	Communicational (medium): both use the user table	Separate authentication and history into two classes if module grows

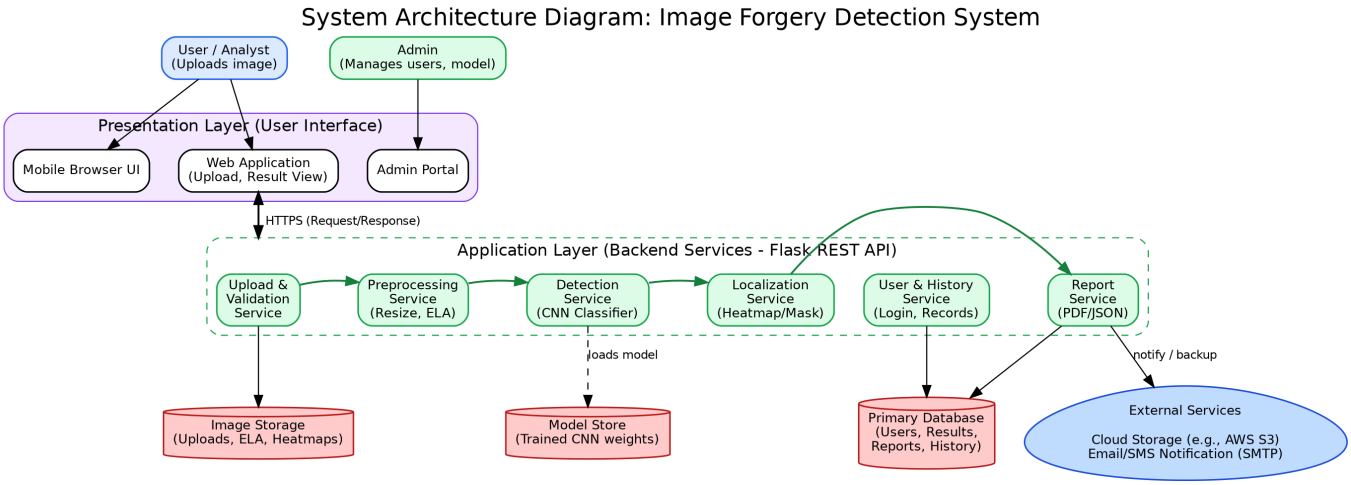
G. Module Interaction and Design Quality

Sequence diagram showing how the major modules exchange information for one image analysis:



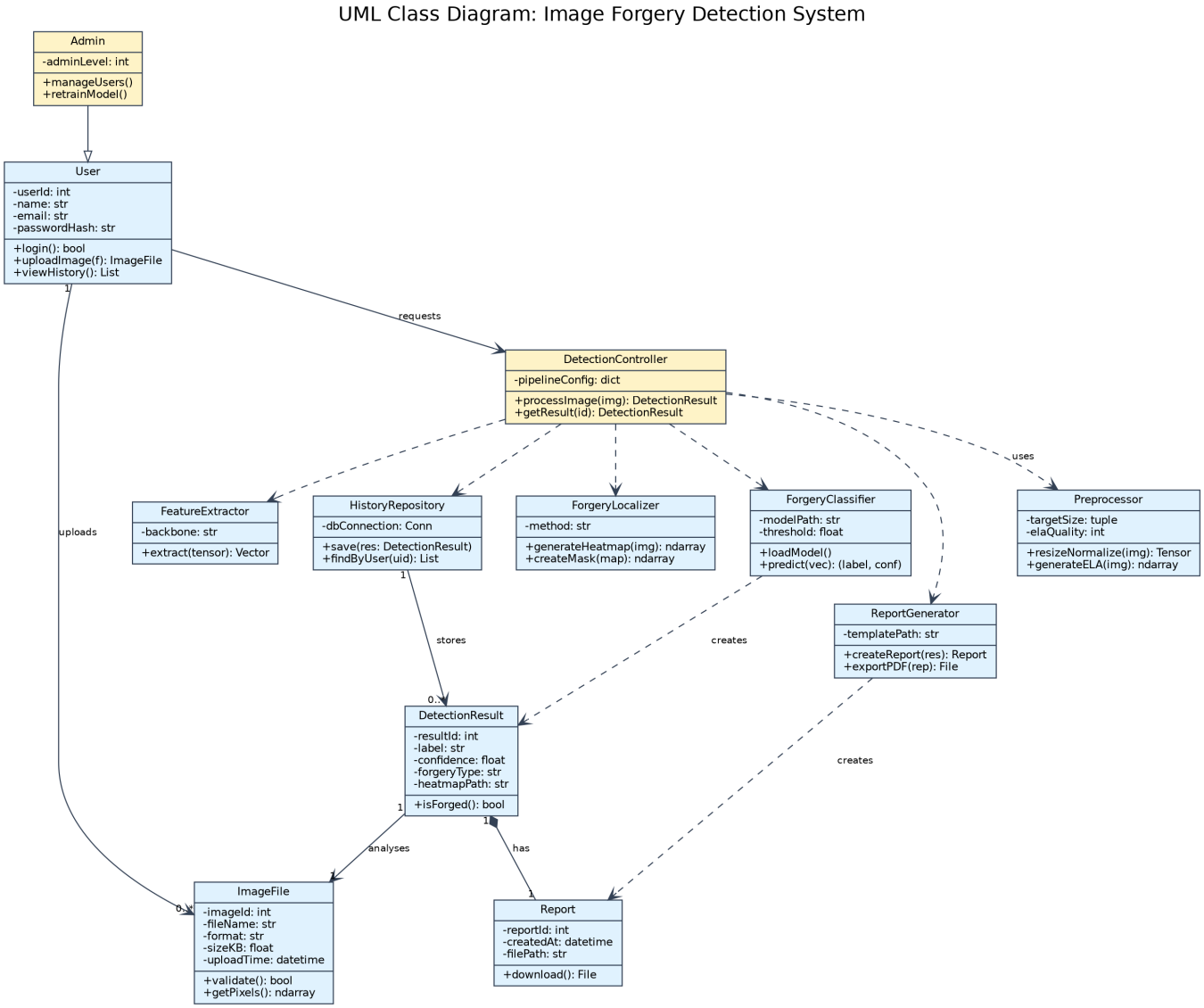
Design quality summary: modules communicate in a single direction along the pipeline, coupling is mostly data coupling, and every module has one clear responsibility (functional or sequential cohesion). This makes the system easy to test, maintain and extend (e.g., adding a new detector for splicing only changes M3/M4).

Annexure 1: System Architecture Diagram



Legend: rounded box = component/service; cylinder = database/storage; ellipse = external service; solid arrow = data flow; dashed arrow = dependency; dashed boundary = application layer.

Annexure 2: UML Class Diagram



Annexure 3: UML Activity Diagram

